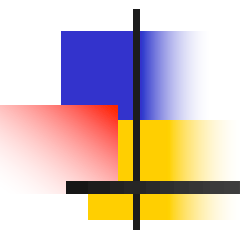
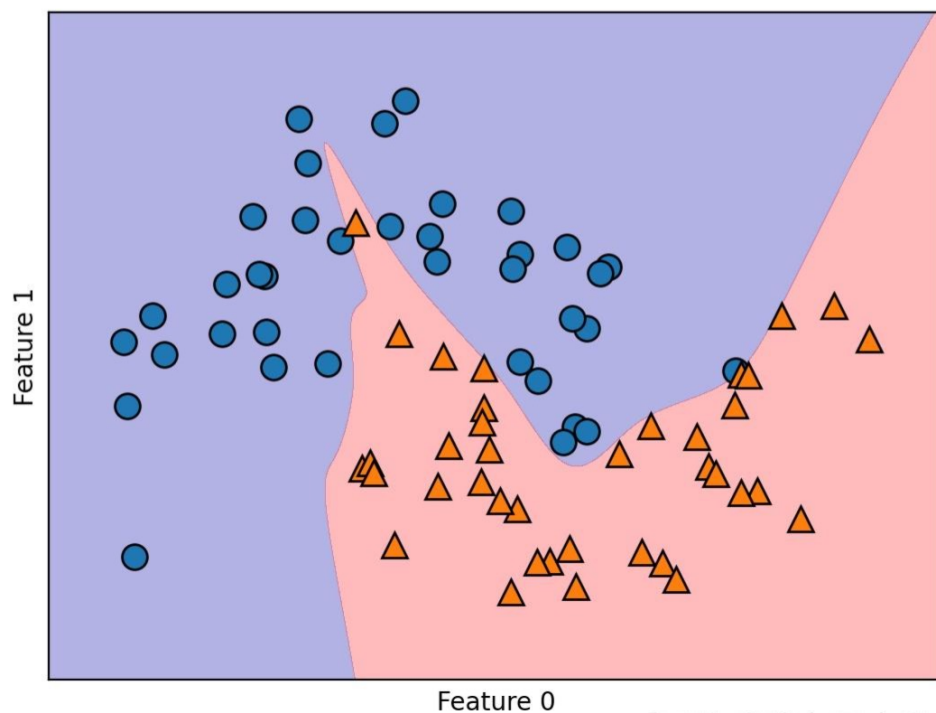


第六章 典型的非线性 分类器



6.1 引言

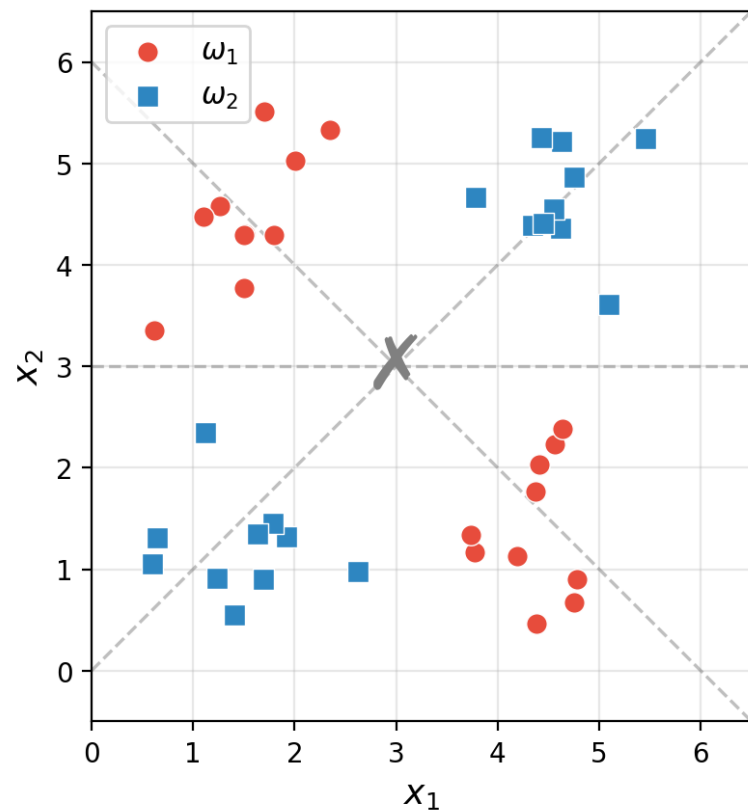
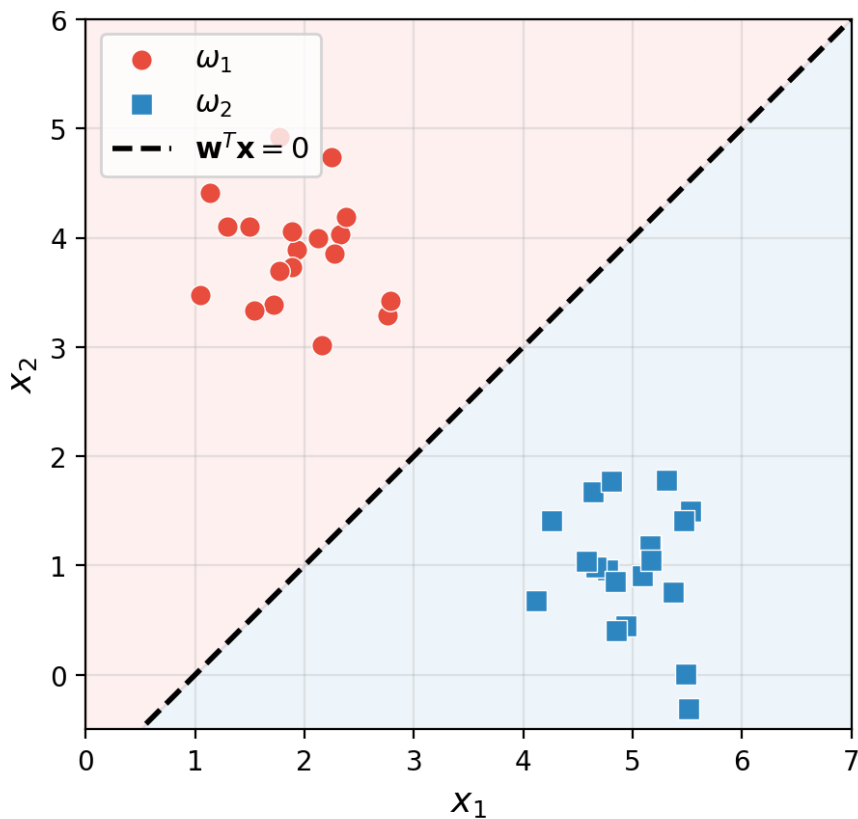
- 很多实际情况下，类别之间的分类边界并不是线性的
- 非线性判别函数并不是明确的一类函数，而是除线性函数外的各种函数的集合

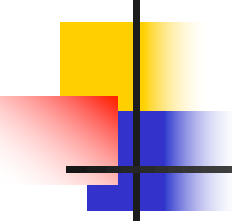


回顾 5.5 感知器

■ 样本集可分性

■ 左侧线性可分，右侧线性不可分





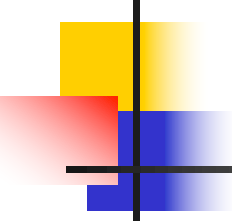
回顾 5.5 感知器

■ 感知器准则函数

- 对于权重向量 $\mathbf{w}$ ，如果某个样本 $\mathbf{x}_i$ 被错误分类，则 $\mathbf{w}^T \mathbf{x}'_i \leq 0$
- 利用对所有错分样本求和作为对其的惩罚：

$$J_P(\mathbf{w}) = \sum_{\mathbf{w}^T \mathbf{x}'_i \leq 0} (-\mathbf{w}^T \mathbf{x}'_i) \quad (5-56)$$

- 当且仅当 $J_p(\mathbf{w}^*) = \min J_p(\mathbf{w}) = 0$ 时 $\mathbf{w}^*$ 是解向量 (⚠️ 等于 0 是因为前面假设了样本集线性可分)
- 利用梯度下降法迭代求解



回顾 5.5 感知器

- 梯度下降

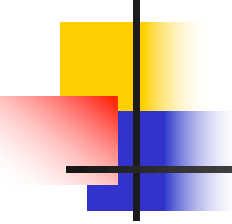
- 参数更新公式:

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \rho_t \nabla J_p(\boldsymbol{w}) \quad (5-57)$$

- $\boldsymbol{w}_{t+1}$: 更新后的下一时刻权重向量
 - $\boldsymbol{w}_t$: 更新前的当前时刻权重向量
 - ρ_t : 步长, 学习率
 - $\nabla J_p(\boldsymbol{w})$: 准则函数 J_p 对 $\boldsymbol{w}$ 求偏导

- 偏导计算:

$$\nabla J_P(\boldsymbol{w}) = \frac{\partial J_P(\boldsymbol{w})}{\partial \boldsymbol{w}} = \frac{\partial \sum_{\boldsymbol{w}^T \boldsymbol{x}'_i \leq 0} (-\boldsymbol{w}^T \boldsymbol{x}'_i)}{\partial \boldsymbol{w}} = \sum_{\boldsymbol{w}^T \boldsymbol{x}'_i \leq 0} (-\boldsymbol{x}'_i) \quad (5-58)$$



回顾 5.5 感知器

- 梯度下降

- 将公式 5-58 代入 公式 5-57:

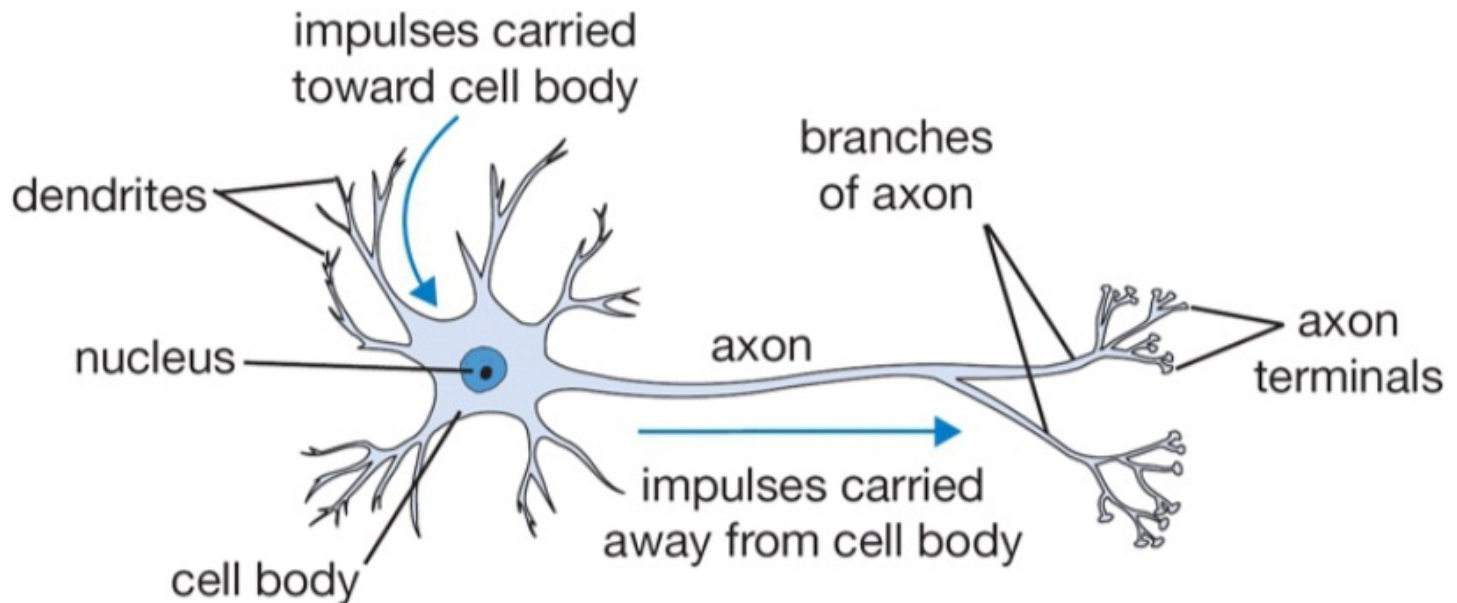
$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t + \rho_t \sum_{\boldsymbol{w}^T \boldsymbol{x}'_i \leq 0} \boldsymbol{x}'_i \quad (5-59)$$

- 上述做法是一次性将所有错误样本都进行修正
 - 更常用的是每次只修正一个样本的固定增量法

6.4.1 神经元感知器

■ 神经元

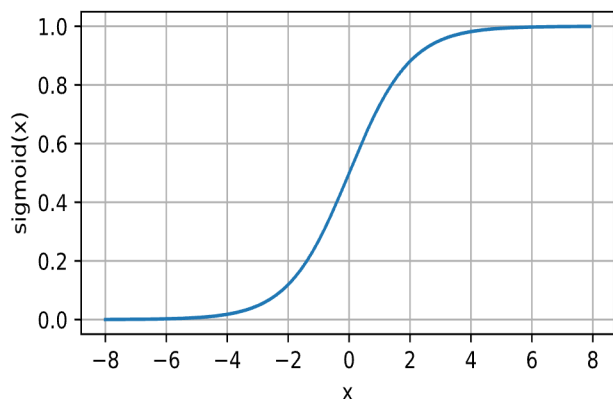
- 是一个神经细胞，是神经系统的基本组成单位
- 由细胞体、树突、轴突、突触构成
- 神经元之间通过突触连接，构成了复杂的神经网络系统



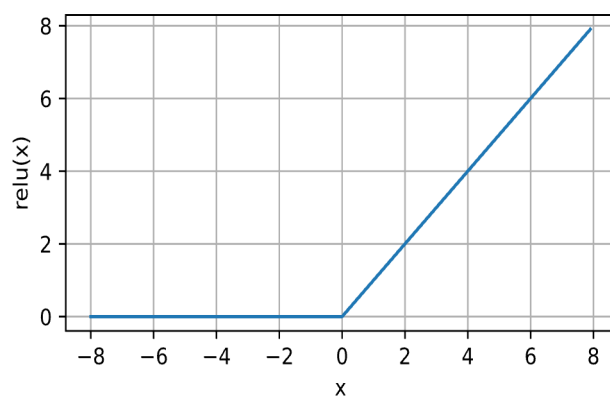
6.4.1 神经元感知器

■ 感知器神经元

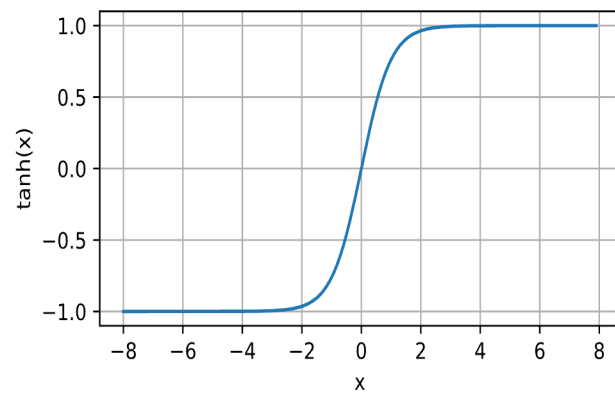
- 输入： d 维输入信号 $x = [x_1, x_2, \dots, x_d]^T$
- 待学习参数： d 维待优化权重向量 $w = [w_1, w_2, \dots, w_d]^T$ 和一维待优化偏置项 b
- 非线性激活函数 $\theta(\cdot)$, 可以是 Sigmoid、ReLU、Tanh 函数等



Sigmoid



ReLU



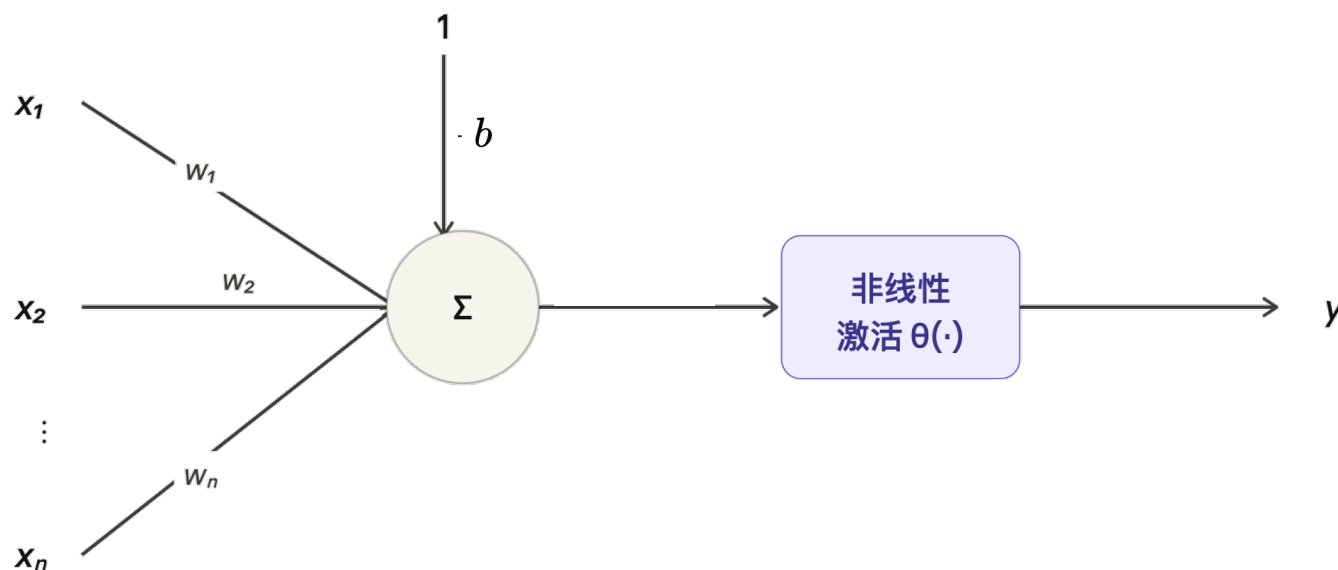
Tanh

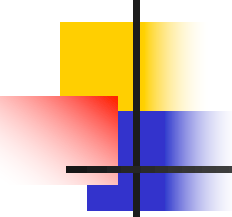
6.4.1 神经元感知器

- 感知器神经元

- 公式表示：

$$y = \theta \left(\sum_{i=1}^n w_i x_i + b \right) \quad (6-14)$$





6.4.2 多个感知器实现非线性分类

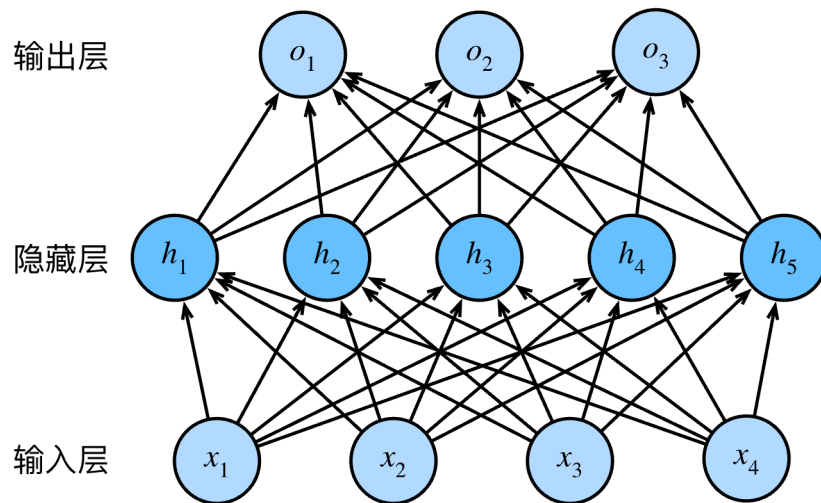
- 单个感知器神经元的局限
 - 仅能完成线性可分数据的分类问题
 - 无法解决非线性问题
- 改进：多层感知机（前馈神经网络）
 - 前一层神经元的输出是后一层神经元的输入
 - 是一种可普遍适用的非线性学习机器，能够实现任意复杂的函数映射

6.4.2 多个感知器实现非线性分类

- 多层感知机

- 构成

- **输入层**：每个节点对应输入样本 x 的每一维，负责将把每一维的信号分发给隐藏层
 - **隐藏层**：每层由多个感知器神经元组成，每个神经元根据公式 6-14 对上一层输入进行处理 (⚠️ 可以有多层)

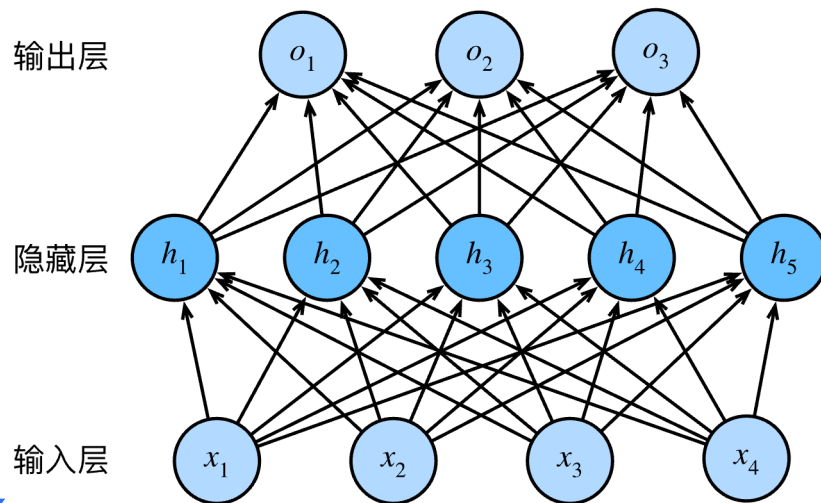


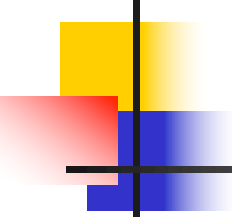
6.4.2 多个感知器实现非线性分类

- 多层感知机

- 构成

- 输出层：对于二分类任务，输出层通常只有 1 个神经元 (经过 Sigmoid 函数使输出值介于 0-1 之间)；对于多分类问题 (c 类)，输出层存在 c 个神经元 (经过 Softmax 函数保证所有类别输出概率之和为 1)





6.4.2 多个感知器实现非线性分类

- 为什么要引入隐藏层？
 - **模型复杂性和表示能力**。隐藏层增加了网络的容量，使其能够学习并表示更复杂的函数
 - **层次化特征学习**。例如，在图像识别中，第一层可能会学习边缘和纹理，第二层可能会学习形状和模式
 - **非线性堆叠**。通过引入多个隐藏层并在每个层之后应用非线性激活函数，网络可以表示更复杂的非线性函数
- 隐藏层与输出层的关键区别
 - 隐藏层的激活函数 (ReLU、Sigmoid、Tanh等) 侧重于引入非线性表达能力
 - 输出层的激活函数 (Sigmoid、Softmax) 用于概率计算

6.4.2 多个感知器实现非线性分类

■ 符号定义

- d 维输入向量 $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$
- 上标 l 表示神经元节点所在的层数，输入层记 $l = 0$
- 总层数记作 L
- 输出层节点的个数为 m ，输出层输出记作 $\mathbf{y} = [y_1, y_2, \dots, y_m]^T$
- 第 l 层的神经元个数记作 n_l ，对于输入层有 $n_0 = d$ ，对于输出层有 $n_L = m$

6.4.2 多个感知器实现非线性分类

符号定义

- 第 l 层的待优化权重记作 $w_{ij}^{(l)}$, 其中下标 ij 表示第 $l-1$ 层的节点 i 连接到第 l 层的节点 j 的权重
- 第 l 层中节点 j 的待优化偏置项记作 $b_j^{(l)}$
- 第 l 层中节点 j 的净输入 (加权求和加偏置后, 未经过激活函数之前的值):

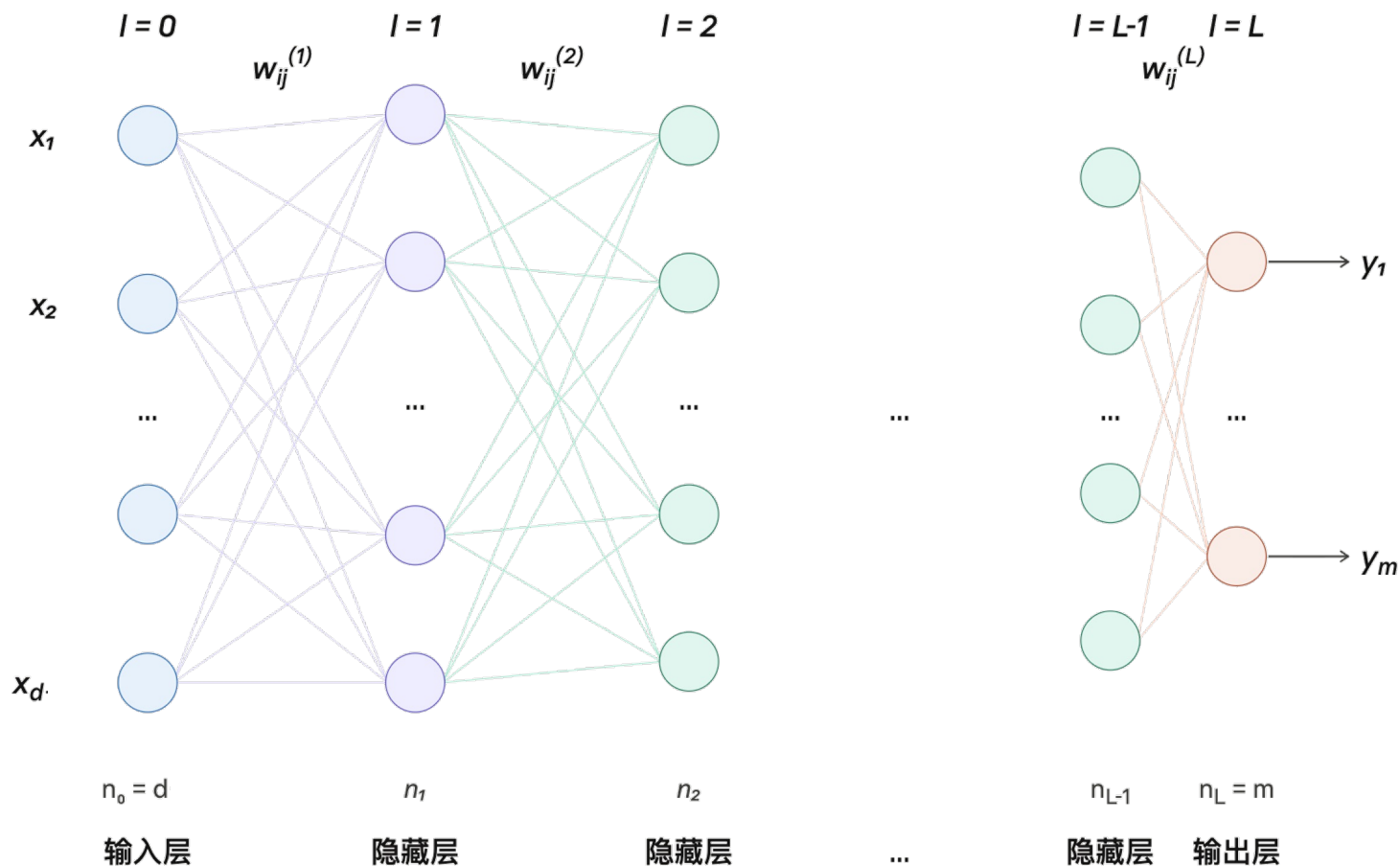
$$\text{net}_j^{(l)} = \sum_{i=1}^{n_{l-1}} w_{ij}^{(l)} x_i^{(l-1)} + b_j^{(l)}$$

- 第 l 层中的第 i 个神经元的输出 $x_i^{(l)}$, 对于输入层有 $x_i^{(0)} = x_i, i = 1, 2, \dots, d$, 对于隐藏层:

$$x_j^{(l)} = \theta(\text{net}_j^{(l)})$$

6.4.2 多个感知器实现非线性分类

■ 示意图



6.4.2 多个感知器实现非线性分类

- 参数量统计

- 除输入层外，对于第 l 层，其参数包括两部分：

- 权重矩阵 $W^{(l)}$ ：连接第 $l-1$ 层到第 l 层，维度为 $\mathbb{R}^{n_{l-1} \times n_l}$ ，参数个数为 $n_{l-1} \times n_l$
 - 偏置向量 $b^{(l)}$ ：第 l 层每个节点一个偏置，参数个数为 n_l
 - 第 l 层的参数总数为：

$$P_l = n_{l-1} \times n_l + n_l = (n_{l-1} + 1)n_l$$

6.4.2 多个感知器实现非线性分类

- 参数量统计

- 整个模型的参数量

- ⚠ 输入层 ($l = 0$) 不含可学习参数, 参数仅存在于 $l = 1, 2, \dots, L$ 层。将各层参数求和:

$$P_{\text{total}} = \sum_{l=1}^L (n_{l-1} + 1)n_l$$

- 其中, 输入层 $n_0 = d$, 输出层 $n_L = m$

6.4.2 多个感知器实现非线性分类

■ 前向传播

- 输入数据被馈送到输入层，然后传递到隐藏层，并最终生成输出层的输出

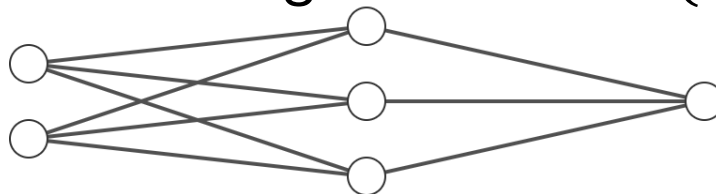
- 计算示例

- 输入样本: $\mathbf{x} = [x_1, x_2]^T = [1, 1]^T$

- 隐藏层: 1 层, 节点数 $n_1 = 3$

- 输出层: 节点数 $n_2 = 1$

- 激活函数: Sigmoid 函数 $\theta(z) = \frac{1}{1+e^{-z}}$



Input Layer $\in \mathbb{R}^2$

Hidden Layer $\in \mathbb{R}^3$

Output Layer $\in \mathbb{R}^1$

6.4.2 多个感知器实现非线性分类

- 前向传播

- 计算示例

- 隐藏层初始化权重矩阵与偏置向量：

$$\mathbf{W}^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} \end{pmatrix} = \begin{pmatrix} 1 & 1 & -1 \\ -1 & 1 & 1 \end{pmatrix}, \quad \mathbf{b}^{(1)} = \begin{pmatrix} 0 \\ -1 \\ -1 \end{pmatrix}$$

- 输出层初始化权重向量与偏置：

$$\mathbf{W}^{(2)} = \begin{pmatrix} w_{11}^{(2)} \\ w_{21}^{(2)} \\ w_{31}^{(2)} \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix}, \quad b^{(2)} = -1$$

6.4.2 多个感知器实现非线性分类

■ 前向传播

■ 计算示例：第一步 输入层 → 隐藏层

■ 隐藏层第一层节点 $j = 1$:

$$\text{net}_1^{(1)} = w_{11}^{(1)} x_1 + w_{21}^{(1)} x_2 + b_1^{(1)} = 1 \times 1 + (-1) \times 1 + 0 = 0$$

$$x_1^{(1)} = \theta(0) = \frac{1}{1 + e^0} = \frac{1}{2} = 0.5$$

■ 隐藏层第一层节点 $j = 2$:

$$\text{net}_2^{(1)} = w_{12}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)} = 1 \times 1 + 1 \times 1 + (-1) = 1$$

$$x_2^{(1)} = \theta(1) = \frac{1}{1 + e^{-1}} \approx 0.7311$$

■ 隐藏层第一层节点 $j = 3$:

$$\text{net}_3^{(1)} = (-1) \times 1 + 1 \times 1 + (-1) = -1 \quad x_3^{(1)} = \theta(-1) = \frac{1}{1 + e^1} \approx 0.2689$$

6.4.2 多个感知器实现非线性分类

■ 前向传播

■ 计算示例：第二步 隐藏层 → 输出层

$$\text{net}_1^{(2)} = w_{11}^{(2)} x_1^{(1)} + w_{21}^{(2)} x_2^{(1)} + w_{31}^{(2)} x_3^{(1)} + b^{(2)}$$

$$= 2 \times 0.5 + 1 \times \theta(1) + 1 \times \theta(-1) + (-1)$$

$$= 1 + \underbrace{[\theta(1) + \theta(-1)]}_{=1} - 1 = 1$$

Sigmoid 函数性质

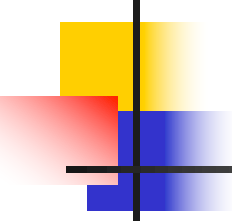
$$\theta(-z) = 1 - \theta(z)$$

$$y = \theta(1) \approx 0.7311$$

■ 分类决策

$$y = 0.7311 > 0.5 \implies \text{判定属于 } \omega_1 \text{ 类}$$

⚠ 如果样本 x 不属于 ω_1 类而属于 ω_2 类，则需要对待优化权重矩阵与偏置向量进行更新



6.4.3 反向传播算法

- 基本概念

- 平方误差损失:

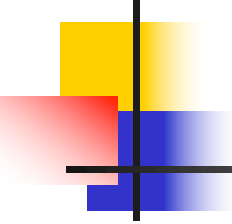
$$E = \frac{1}{2}(y - t)^2$$

- $t \in \{0,1\}$ 是目标标签 (ω_1 类对应标签为 $t = 1$, ω_2 类对应标签为 $t = 0$)

- Sigmoid 函数求导:

$$\theta(z) = \frac{1}{1 + e^{-z}}$$

$$\theta'(z) = \theta(z)(1 - \theta(z))$$



6.4.3 反向传播算法

- 基本概念

- 链式求导法则

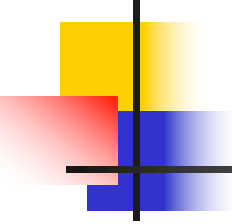
- 核心思想

- 复合函数的导数等于外层函数对中间变量的导数，乘以中间变量对自变量的导数

- 多层复合公式

- 若 $y = f(u), u = g(v), v = h(x)$ ，则：

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dv} \cdot \frac{dv}{dx}$$



6.4.3 反向传播算法

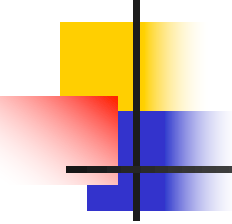
- 基本概念

- 链式求导法则

- 简单示例

- 设 $y = (3x^2 + 1)^5$, 令 $u = 3x^2 + 1$, 则 $y = u^5$:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx} = 5u^4 \cdot 6x = 30x(3x^2 + 1)^4$$



回顾 5.5 感知器

- 梯度下降

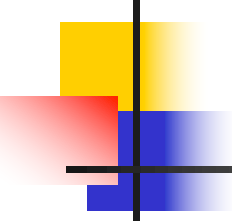
- 参数更新公式:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \rho_t \nabla J_p(\mathbf{w}) \quad (5-57)$$

- $\mathbf{w}_{t+1}$: 更新后的下一时刻权重向量
 - $\mathbf{w}_t$: 更新前的当前时刻权重向量
 - ρ_t : 步长, 学习率
 - $\nabla J_p(\mathbf{w})$: 准则函数 J_p 对 $\mathbf{w}$ 求偏导

- 偏导计算:

$$\nabla J_P(\mathbf{w}) = \frac{\partial J_P(\mathbf{w})}{\partial \mathbf{w}} = \frac{\partial \sum_{\mathbf{w}^T \mathbf{x}'_i \leq 0} (-\mathbf{w}^T \mathbf{x}'_i)}{\partial \mathbf{w}} = \sum_{\mathbf{w}^T \mathbf{x}'_i \leq 0} (-\mathbf{x}'_i) \quad (5-58)$$



6.4.3 反向传播算法

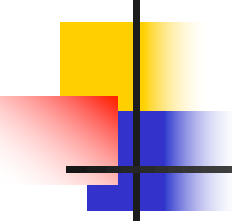
■ 训练目标

- 给定训练样本，最小化平方误差损失函数 E
- 需要计算出每个参数的梯度：

$$\frac{\partial E}{\partial w_{ij}^{(l)}}, \quad \frac{\partial E}{\partial b_j^{(l)}}, \quad l = 1, 2, \dots, L$$

■ 算法思想

- 利用链式求导法则，从输出层反向逐层计算梯度
- 一次前向 + 一次反向，即可得到所有参数的梯度



6.4.3 反向传播算法

- 误差信号 δ

- 定义

- 第 l 层中神经元 j 的误差信号，即损失对该节点净输入的偏导数：

$$\delta_j^{(l)} = \frac{\partial E}{\partial \text{net}_j^{(l)}}$$

- 其中， $\text{net}_j^{(l)}$ 是第 l 层中节点 j 的净输入（加权求和加偏置后，未经过激活函数之前的值）：

$$\text{net}_j^{(l)} = \sum_{i=1}^{n_{l-1}} w_{ij}^{(l)} x_i^{(l-1)} + b_j^{(l)}$$

6.4.3 反向传播算法

- 误差信号 δ

- 为什么要定义 $\delta_j^{(l)}$?

- 一旦得到 $\delta_j^{(l)}$, 第 l 层中神经元 j 的全部参数梯度立刻可以计算

- 由链式法则:

$$\frac{\partial E}{\partial w_{ij}^{(l)}} = \underbrace{\frac{\partial E}{\partial \text{net}_j^{(l)}}}_{\delta_j^{(l)}} \underbrace{\frac{\partial \text{net}_j^{(l)}}{\partial w_{ij}^{(l)}}}, \quad \frac{\partial E}{\partial b_j^{(l)}} = \underbrace{\frac{\partial E}{\partial \text{net}_j^{(l)}}}_{\delta_j^{(l)}} \cdot \underbrace{\frac{\partial \text{net}_j^{(l)}}{\partial b_j^{(l)}}}$$

6.4.3 反向传播算法

- 误差信号 δ

- 为什么要定义 $\delta_j^{(l)}$?

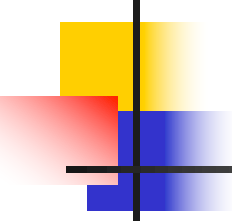
- 由 $\text{net}_j^{(l)} = \sum_{i=1}^{n_{l-1}} w_{ij}^{(l)} x_i^{(l-1)} + b_j^{(l)}$ 可得:

$$\frac{\partial \text{net}_j^{(l)}}{\partial w_{ij}^{(l)}} = x_i^{(l-1)}, \quad \frac{\partial \text{net}_j^{(l)}}{\partial b_j^{(l)}} = 1$$

- 由此:

$$\frac{\partial L}{\partial w_{ij}^{(l)}} = \delta_j^{(l)} x_i^{(l-1)}, \quad \frac{\partial L}{\partial b_j^{(l)}} = \delta_j^{(l)}$$

⚠ 反向传播的核心任务就是逐层计算 $\delta_j^{(l)}$



6.4.3 反向传播算法

- 输出层的 δ

- 本例输出层只有一个神经元，平方误差损失：

$$E = \frac{1}{2}(y - t)^2$$

- 根据链式求导法则：

$$\delta_1^{(L)} = \frac{\partial E}{\partial \text{net}_1^{(L)}} = \boxed{\frac{\partial E}{\partial y}} \cdot \frac{\partial y}{\partial \text{net}_1^{(L)}}$$

- 第一项：

$$\frac{\partial E}{\partial y} = y - t$$

6.4.3 反向传播算法

- 输出层的 δ

- 本例输出层只有一个神经元，平方误差损失：

$$E = \frac{1}{2}(y - t)^2$$

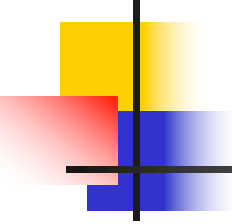
- 根据链式求导法则：

$$\delta_1^{(L)} = \frac{\partial E}{\partial \text{net}_1^{(L)}} = \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial \text{net}_1^{(L)}}$$

- 第二项，根据 $y = \theta(\text{net}_1^{(L)})$ ：

$$\frac{\partial y}{\partial \text{net}_1^{(L)}} = \theta'(\text{net}_1^{(L)}) = \theta(\text{net}_1^{(L)})(1 - \theta(\text{net}_1^{(L)})) = y(1 - y)$$

Sigmoid 函数求导公式



6.4.3 反向传播算法

- 输出层的 δ

- 本例输出层只有一个神经元，平方误差损失：

$$E = \frac{1}{2}(y - t)^2$$

- 根据链式求导法则：

$$\delta_1^{(L)} = \frac{\partial E}{\partial \text{net}_1^{(L)}} = \boxed{\frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial \text{net}_1^{(L)}}}$$

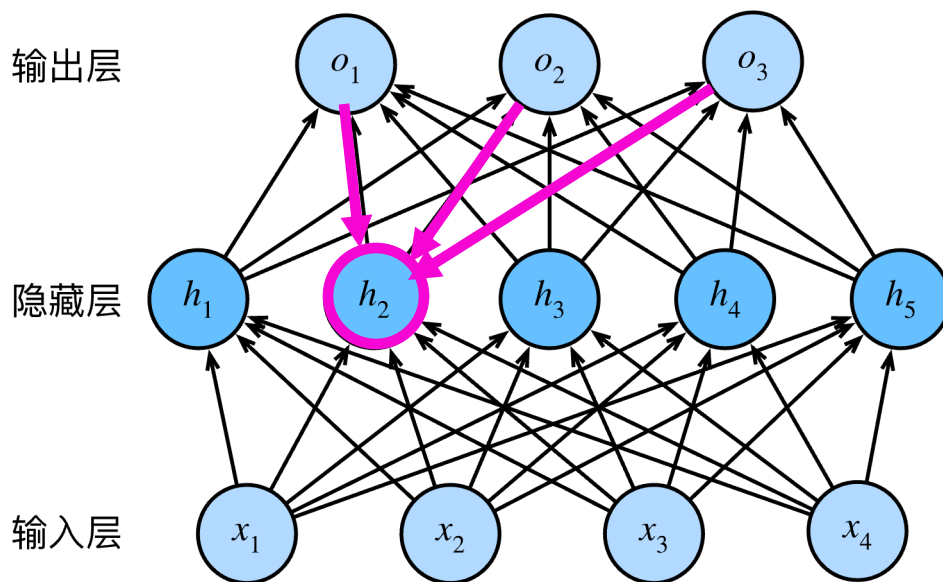
- 合并：

$$\delta_1^{(L)} = (y - t)y(1 - y)$$

6.4.3 反向传播算法

■ 隐藏层的 δ

- 第 l 层中节点 j 对平方误差损失 E 的影响通过下一层所有节点间接传递
- 需要从第 $l + 1$ 层将误差信号传递回来



6.4.3 反向传播算法

■ 隐藏层的 δ

- 第 l 层中节点 j 对平方误差损失 E 的影响通过下一层所有节点间接传递
- 需要从第 $l + 1$ 层将误差信号传递回来
- 根据链式求导法则：

$$\delta_j^{(l)} = \frac{\partial E}{\partial \text{net}_j^{(l)}} = \sum_{k=1}^{n_{l+1}} \boxed{\frac{\partial E}{\partial \text{net}_k^{(l+1)}}} \cdot \frac{\partial \text{net}_k^{(l+1)}}{\partial \text{net}_j^{(l)}}$$

- 第一项是第 $l + 1$ 层中神经元 k 的误差信号，即 $\delta_k^{(l+1)}$

6.4.3 反向传播算法

- 隐藏层的 δ

- 根据链式求导法则：

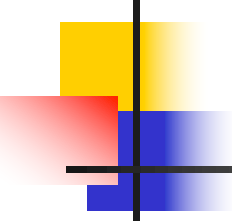
$$\delta_j^{(l)} = \frac{\partial E}{\partial \text{net}_j^{(l)}} = \sum_{k=1}^{n_{l+1}} \frac{\partial E}{\partial \text{net}_k^{(l+1)}} \cdot \frac{\partial \text{net}_k^{(l+1)}}{\partial \text{net}_j^{(l)}}$$

- 第二项：

$$\frac{\partial \text{net}_k^{(l+1)}}{\partial \text{net}_j^{(l)}} = \frac{\partial \text{net}_k^{(l+1)}}{\partial x_j^{(l)}} \cdot \frac{\partial x_j^{(l)}}{\partial \text{net}_j^{(l)}} = w_{jk}^{(l+1)} \cdot \theta'(\text{net}_j^{(l)})$$

第 l 层中节点 j 的输出

$$\text{net}_j^{(l+1)} = \sum_{i=1}^{n_l} w_{ij}^{(l+1)} x_i^{(l)} + b_j^{(l+1)}$$



6.4.3 反向传播算法

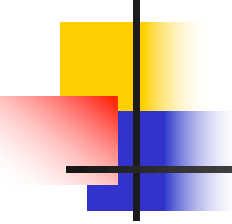
- 隐藏层的 δ

- 两项合并:

$$\delta_j^{(l)} = \left(\sum_{k=1}^{n_{l+1}} w_{jk}^{(l+1)} \delta_k^{(l+1)} \right) \theta'(\text{net}_j^{(l)})$$

- 公式表明:

- 在前向传播过程中, 第 l 层中节点 j 通过权重 $w_{jk}^{(l+1)}$ 将信号分发给下一层所有节点 k
 - 在反向传播过程中, 第 $l+1$ 层中各节点的误差 $\delta_k^{(l+1)}$ 沿相同的权重 $w_{jk}^{(l+1)}$ 汇聚回第 l 层节点 j

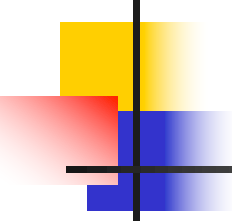


6.4.3 反向传播算法

- 多层感知机算法流程

- 输入：训练样本 (x, t) ，当前参数 $\{w_{ij}^{(l)}, b_j^{(l)}\}$ ，学习率 ρ
- 步骤 1：前向传播。逐层计算并保存所有 $\text{net}_j^{(l)}$ 与 $x_j^{(l)}$ ，直到得到输出 y
- 步骤 2：根据输出 y 与样本标签 t 计算损失 E
- 步骤 3：计算输出层误差信号 (⚠ 单输出神经元情形)

$$\delta_1^{(L)} = (y - t)y(1 - y)$$



6.4.3 反向传播算法

■ 算法流程

- 步骤 4: 逐层计算隐藏层误差信号

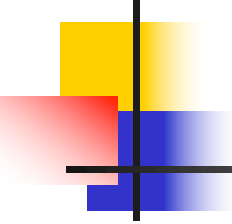
$$\delta_j^{(l)} = \left(\sum_{k=1}^{n_{l+1}} w_{jk}^{(l+1)} \delta_k^{(l+1)} \right) \theta' \left(\text{net}_j^{(l)} \right)$$

- 步骤 5: 计算所有参数梯度:

$$\frac{\partial E}{\partial w_{ij}^{(l)}} = \delta_j^{(l)} x_i^{(l-1)}, \quad \frac{\partial E}{\partial b_j^{(l)}} = \delta_j^{(l)}$$

- 步骤 6: 梯度下降法进行参数更新

$$w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} - \rho \frac{\partial E}{\partial w_{ij}^{(l)}}, \quad b_j^{(l)} \leftarrow b_j^{(l)} - \rho \frac{\partial E}{\partial b_j^{(l)}}$$



6.4.3 反向传播算法

- 计算示例

- 示例设定回顾

- 隐藏层初始化权重矩阵与偏置向量：

$$\mathbf{W}^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} \end{pmatrix} = \begin{pmatrix} 1 & 1 & -1 \\ -1 & 1 & 1 \end{pmatrix}, \quad \mathbf{b}^{(1)} = \begin{pmatrix} 0 \\ -1 \\ -1 \end{pmatrix}$$

- 输出层初始化权重向量与偏置：

$$\mathbf{W}^{(2)} = \begin{pmatrix} w_{11}^{(2)} \\ w_{21}^{(2)} \\ w_{31}^{(2)} \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix}, \quad b^{(2)} = -1$$

6.4.3 反向传播算法

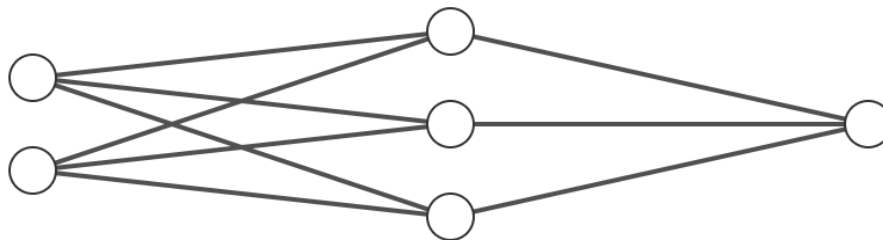
- 计算示例

- 示例设定回顾

隐藏层 { $\left. \begin{array}{l} \text{net}_1^{(1)} = 0, \quad \text{net}_2^{(1)} = 1, \quad \text{net}_3^{(1)} = -1 \\ x_1^{(1)} = 0.5, \quad x_2^{(1)} \approx 0.7311, \quad x_3^{(1)} \approx 0.2689 \end{array} \right\}$

输出层

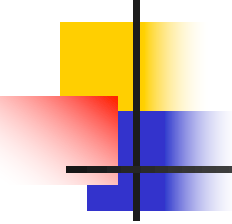
$$\text{net}_1^{(2)} = 1, \quad y \approx 0.7311$$



Input Layer $\in \mathbb{R}^2$

Hidden Layer $\in \mathbb{R}^3$

Output Layer $\in \mathbb{R}^1$



6.4.3 反向传播算法

- 计算示例

- 标签约定

- $t = 1 \Leftrightarrow \omega_1$ 类, $t = 0 \Leftrightarrow \omega_2$ 类

- 样本真实属于 ω_2 类, 但由于前向输出 $y \approx 0.7311 > 0.5$ 被误判为 ω_1 类, 需要更新参数

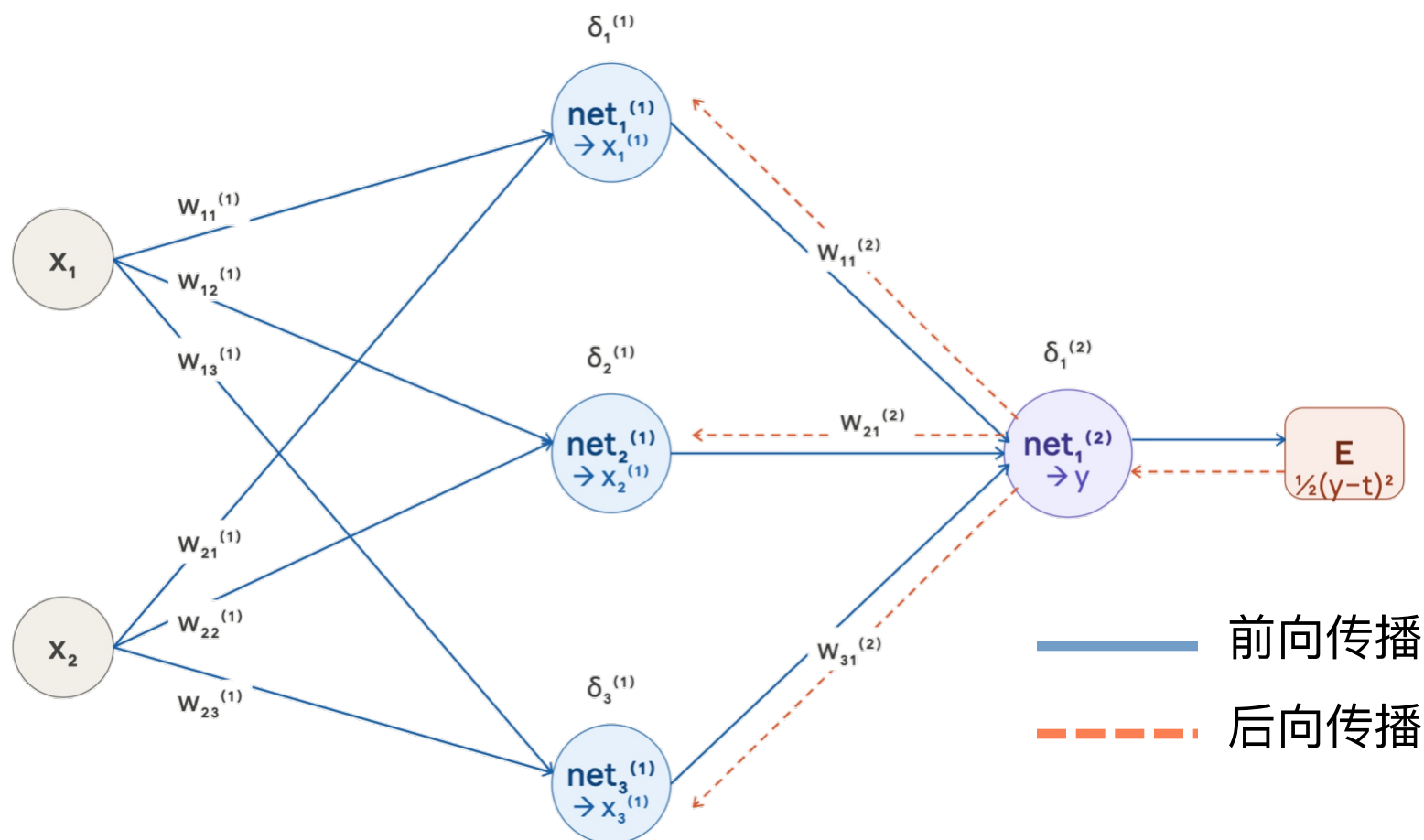
- 学习率 $\rho = 1$

- 损失函数

$$E = \frac{1}{2}(y - t)^2 = \frac{1}{2}(0.7311 - 0)^2 \approx 0.2673$$

6.4.3 反向传播算法

■ 计算示例



6.4.3 反向传播算法

■ 计算示例

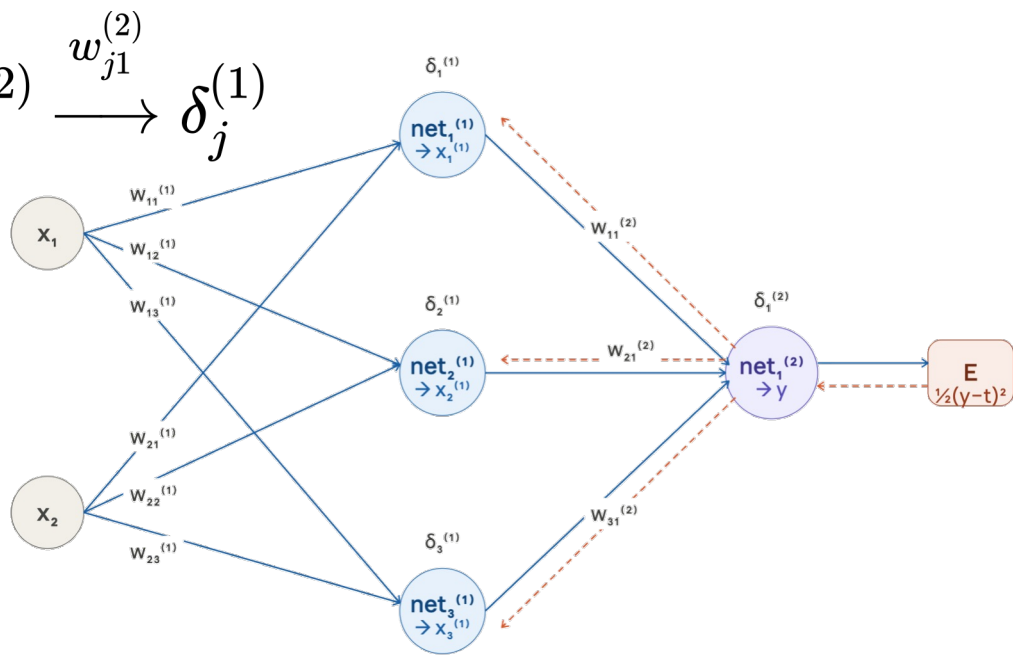
■ 前向传播 (蓝色箭头)

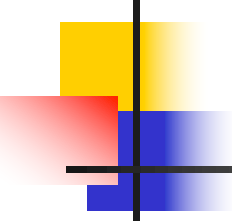
$$\mathbf{x} \xrightarrow{w^{(1)}, b^{(1)}} \text{net}^{(1)} \rightarrow \mathbf{x}^{(1)} \xrightarrow{w^{(2)}, b^{(2)}} \text{net}_1^{(2)} \rightarrow y \rightarrow E$$

■ 后向传播 (橙色虚线)

$$E \rightarrow \delta_1^{(2)} \xrightarrow{w_{j1}^{(2)}} \delta_j^{(1)}$$

- **!** 一旦 δ 全部算出, $\frac{\partial E}{\partial w}$ 和 $\frac{\partial E}{\partial b}$ 可以直接计算出





6.4.3 反向传播算法

- 计算示例

- 计算输出层误差信号 $\delta_1^{(2)}$:

$$\delta_1^{(2)} = (y - t)y(1 - y)$$

- 代入数值 ($y \approx 0.7311, t = 0$):

$$\begin{aligned}\delta_1^{(2)} &= (0.7311 - 0) \times 0.7311 \times (1 - 0.7311) \\ &= 0.7311 \times 0.7311 \times 0.2689 \approx 0.1437\end{aligned}$$

6.4.3 反向传播算法

■ 计算示例

- 计算隐藏层误差信号 $\delta_j^{(1)}$:

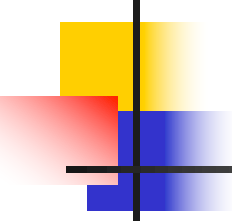
-  下一层只有一个节点，求和退化为单项

$$\delta_j^{(1)} = w_{j1}^{(2)} \delta_1^{(2)} \cdot x_j^{(1)} (1 - x_j^{(1)}), \quad j = 1, 2, 3$$

- 代入数值：上一步得到的结果

- 节点 $j = 1$ ($w_{11}^{(2)} = 2, x_1^{(1)} = 0.5$)

$$\delta_1^{(1)} = 2 \times 0.1437 \times 0.5 \times 0.5 \approx 0.0719$$



6.4.3 反向传播算法

- 计算示例

- 计算隐藏层误差信号 $\delta_j^{(1)}$:

$$\delta_j^{(1)} = w_{j1}^{(2)} \delta_1^{(2)} \cdot x_j^{(1)} (1 - x_j^{(1)}), \quad j = 1, 2, 3$$

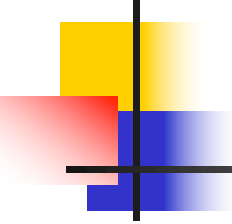
- 代入数值:

- 节点 $j = 2$ ($w_{21}^{(2)} = 1, x_2^{(1)} = 0.7311$)

$$\delta_2^{(1)} = 1 \times 0.1437 \times 0.7311 \times 0.2689 \approx 0.0283$$

- 节点 $j = 3$ ($w_{31}^{(2)} = 1, x_3^{(1)} = 0.2689$)

$$\delta_3^{(1)} = 1 \times 0.1437 \times 0.2689 \times 0.7311 \approx 0.0283$$



6.4.3 反向传播算法

- 计算示例

- 计算输出层参数梯度并进行梯度下降：

$$\frac{\partial E}{\partial w_{i1}^{(2)}} = \delta_1^{(2)} \cdot x_i^{(1)}, \quad \frac{\partial E}{\partial b^{(2)}} = \delta_1^{(2)}$$

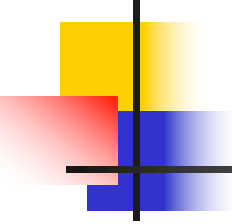
- 代入数值：

$$\frac{\partial E}{\partial w_{11}^{(2)}} = 0.1437 \times 0.5 \approx 0.0719$$

$$\frac{\partial E}{\partial w_{31}^{(2)}} = 0.1437 \times 0.2689 \approx 0.0386$$

$$\frac{\partial E}{\partial w_{21}^{(2)}} = 0.1437 \times 0.7311 \approx 0.1050$$

$$\frac{\partial E}{\partial b^{(2)}} = 0.1437$$



6.4.3 反向传播算法

- 计算示例

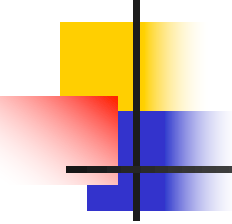
- 计算输出层参数梯度并进行梯度下降：

$$w_{11}^{(2)} \leftarrow 2 - 0.0719 \approx 1.9281$$

$$w_{21}^{(2)} \leftarrow 1 - 0.1050 \approx 0.8950$$

$$w_{31}^{(2)} \leftarrow 1 - 0.0386 \approx 0.9614$$

$$b^{(2)} \leftarrow -1 - 0.1437 \approx -1.1437$$



6.4.3 反向传播算法

- 计算示例

- 计算隐藏层参数梯度并进行梯度下降：

$$\frac{\partial E}{\partial w_{ij}^{(1)}} = \delta_j^{(1)} \cdot x_i, \quad \frac{\partial E}{\partial b_j^{(1)}} = \delta_j^{(1)}$$

- 代入数值：

$$\frac{\partial E}{\partial w_{11}^{(1)}} = \delta_1^{(1)} \cdot x_1 = 0.0719 \times 1 = 0.0719$$

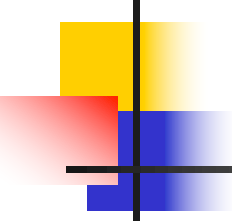
$$\frac{\partial E}{\partial w_{12}^{(1)}} = \delta_2^{(1)} \cdot x_1 = 0.0283 \times 1 = 0.0283$$

$$\frac{\partial E}{\partial w_{13}^{(1)}} = \delta_3^{(1)} \cdot x_1 = 0.0283 \times 1 = 0.0283$$

$$\frac{\partial E}{\partial w_{21}^{(1)}} = \delta_1^{(1)} \cdot x_2 = 0.0719 \times 1 = 0.0719$$

$$\frac{\partial E}{\partial w_{22}^{(1)}} = \delta_2^{(1)} \cdot x_2 = 0.0283 \times 1 = 0.0283$$

$$\frac{\partial E}{\partial w_{23}^{(1)}} = \delta_3^{(1)} \cdot x_2 = 0.0283 \times 1 = 0.0283$$



6.4.3 反向传播算法

- 计算示例

- 计算隐藏层参数梯度并进行梯度下降：

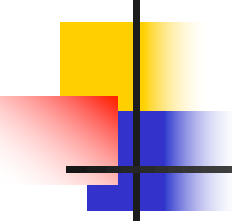
$$\frac{\partial E}{\partial w_{ij}^{(1)}} = \delta_j^{(1)} \cdot x_i, \quad \frac{\partial E}{\partial b_j^{(1)}} = \delta_j^{(1)}$$

- 代入数值：

$$\frac{\partial E}{\partial b_1^{(1)}} = \delta_1^{(1)} = 0.0719$$

$$\frac{\partial E}{\partial b_2^{(1)}} = \delta_2^{(1)} = 0.0283$$

$$\frac{\partial E}{\partial b_3^{(1)}} = \delta_3^{(1)} = 0.0283$$



6.4.3 反向传播算法

- 计算示例

- 计算隐藏层参数梯度并进行梯度下降：

$$w_{11}^{(1)} \leftarrow 1 - 1 \times 0.0719 \approx 0.9281$$

$$w_{12}^{(1)} \leftarrow 1 - 1 \times 0.0283 \approx 0.9717$$

$$w_{13}^{(1)} \leftarrow -1 - 1 \times 0.0283 \approx -1.0283$$

$$w_{21}^{(1)} \leftarrow -1 - 1 \times 0.0719 \approx -1.0719$$

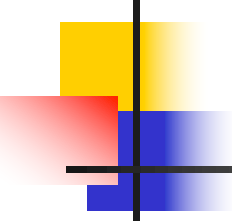
$$w_{22}^{(1)} \leftarrow 1 - 1 \times 0.0283 \approx 0.9717$$

$$w_{23}^{(1)} \leftarrow 1 - 1 \times 0.0283 \approx 0.9717$$

$$b_1^{(1)} \leftarrow 0 - 1 \times 0.0719 \approx -0.0719$$

$$b_2^{(1)} \leftarrow -1 - 1 \times 0.0283 \approx -1.0283$$

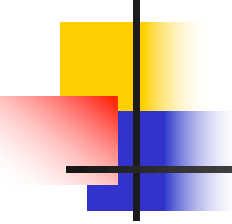
$$b_3^{(1)} \leftarrow -1 - 1 \times 0.0283 \approx -1.0283$$



6.4.3 反向传播算法

■ 例题

- **题 1:** 考虑一个用于手写数字识别的多层感知机，总层数 $L = 4$ ，其网络结构如下：
 - 输入层： $n_0 = d = 16$
 - 第一隐藏层： $n_1 = 20$
 - 第二隐藏层： $n_2 = 10$
 - 第三隐藏层： $n_3 = 8$
 - 输出层： $n_4 = m = 4$
- 计算除输入层外，每一层的参数个数以及整个网络的可训练参数总量。

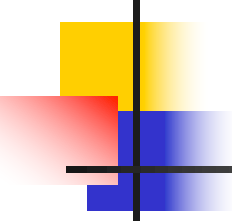


6.4.3 反向传播算法

■ 例题解答

■ 题 1:

层 l	$n_{l-1} \rightarrow n_l$	权重数 $n_{l-1}n_l$	偏置数 n_l	P_l
1	$16 \rightarrow 20$	320	20	340
2	$20 \rightarrow 10$	200	10	210
3	$10 \rightarrow 8$	80	8	88
4	$8 \rightarrow 4$	32	4	36
合计	—	632	42	674



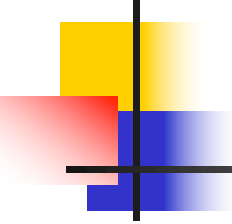
6.4.3 反向传播算法

■ 例题

- 激活函数统一采用 Sigmoid 函数 $\theta(z) = \frac{1}{1+e^{-z}}$

z	-2	-1	0	1	2
$\theta(z)$	0.12	0.27	0.5	0.73	0.88

- **题 2**: 已知某节点的 Sigmoid 输出值为 $\theta(z) = 0.73$, 求 $\theta'(z) = \underline{\hspace{2cm}}$ 。
- **题 3**: 设 $y = \theta(u)$, $u = 2v + 1$, $v = x - 1$, 其中 θ 为 Sigmoid 函数。求 $\frac{dy}{dx}\bigg|_{x=1} = \underline{\hspace{2cm}}$ 。



6.4.3 反向传播算法

■ 例题解答

■ 题 2:

由 $\theta'(z) = \theta(z)(1 - \theta(z))$:

$$\theta'(z) = 0.73 \times 0.27 \approx 0.1971$$

■ 题 3:

$$\frac{dy}{dx} = \theta'(u) \cdot 2 \cdot 1 = 2 \theta'(u)$$

代入 $x = 1 : v = 0, u = 1, \theta'(1) = 0.73 \times 0.27 \approx 0.1971$

$$\left. \frac{dy}{dx} \right|_{x=1} \approx 2 \times 0.1971 \approx 0.3942$$

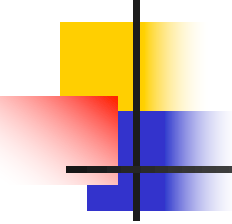
6.4.3 反向传播算法

■ 例题

- 激活函数统一采用 Sigmoid 函数 $\theta(z) = \frac{1}{1+e^{-z}}$

z	-2	-1	0	1	2
$\theta(z)$	0.12	0.27	0.5	0.73	0.88

- **题 4**：网络结构：输入层 2 节点，隐藏层 2 节点，输出层 1 节点。已知： $\mathbf{x} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$, $\mathbf{W}^{(1)} = \begin{pmatrix} 1 & 2 \\ -1 & -1 \end{pmatrix}$, $\mathbf{b}^{(1)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$, $\mathbf{W}^{(2)} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$, $b^{(2)} = 1$, 计算 $\text{net}_1^{(1)} = \underline{\hspace{1cm}}$, $\text{net}_2^{(1)} = \underline{\hspace{1cm}}$, $x_1^{(1)} = \underline{\hspace{1cm}}$, $x_2^{(1)} = \underline{\hspace{1cm}}$, $\text{net}_1^{(2)} = \underline{\hspace{1cm}}$, $y = \underline{\hspace{1cm}}$ 。



6.4.3 反向传播算法

- 例题解答

- 题 4:

- 计算隐藏层净输入:

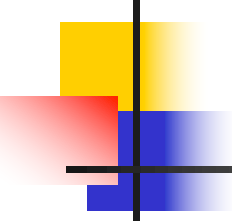
$$\text{net}_1^{(1)} = w_{11}^{(1)}x_1 + w_{21}^{(1)}x_2 + b_1^{(1)} = 1 \times 1 + (-1) \times 1 + 0 = 0$$

$$\text{net}_2^{(1)} = w_{12}^{(1)}x_1 + w_{22}^{(1)}x_2 + b_2^{(1)} = 2 \times 1 + (-1) \times 1 + 0 = 1$$

- 经 Sigmoid 激活得到隐藏层输出:

$$x_1^{(1)} = \theta(0) = 0.5$$

$$x_2^{(1)} = \theta(1) \approx 0.73$$



6.4.3 反向传播算法

- 例题解答

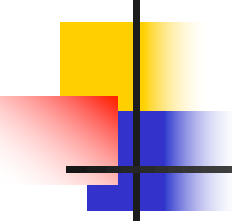
- 题 4:

- 计算输出层净输入:

$$\text{net}_1^{(2)} = w_{11}^{(2)} x_1^{(1)} + w_{21}^{(2)} x_2^{(1)} + b^{(2)} = 1 \times 0.5 + (-1) \times 0.73 + 1 = 0.77$$

- 经 Sigmoid 激活得到输出层输出:

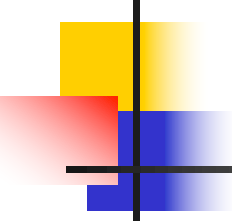
$$y = \theta(0.77) \approx 0.6837$$



6.4.3 反向传播算法

■ 例题

- 激活函数统一采用 Sigmoid 函数 $\theta(z) = \frac{1}{1+e^{-z}}$
- **题 5**：某样本的真实标签 $t = 0$ ，网络输出 $y = 0.88$ 。采用平方误差损失 $E = \frac{1}{2}(y - t)^2 = \underline{\hspace{2cm}}$ 。
- **题 6**：网络结构：输入层 2 节点，隐藏层 2 节点，输出层 1 节点。已知 $y = 0.73, t = 0, x_1^{(1)} = 0.5, x_2^{(1)} = 0.88, w_{11}^{(2)} = 2, w_{21}^{(2)} = -1$ 。计算 $\delta_1^{(2)} = \underline{\hspace{2cm}}, \delta_1^{(1)} = \underline{\hspace{2cm}}, \delta_2^{(1)} = \underline{\hspace{2cm}}$ 。



6.4.3 反向传播算法

- 例题解答

- 题 5:

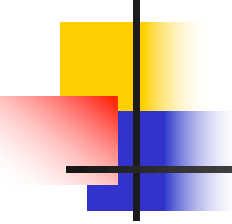
$$E = \frac{1}{2}(y - t)^2 = \frac{1}{2}(0.88 - 0)^2 = \frac{1}{2} \times 0.7744 = 0.3872$$

- 题 6:

- 计算输出层误差信号 $\delta_1^{(2)}$:

$$\delta_1^{(2)} = (y - t)y(1 - y)$$

$$\delta_1^{(2)} = (0.73 - 0) \times 0.73 \times (1 - 0.73) = 0.73 \times 0.73 \times 0.27 \approx 0.1439$$



6.4.3 反向传播算法

■ 例题解答

■ 题 6:

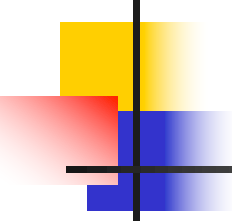
- 计算隐藏层误差信号 $\delta_1^{(1)}$ 和 $\delta_2^{(1)}$:

■  下一层只有一个节点，求和退化为单项

$$\delta_j^{(1)} = w_{j1}^{(2)} \delta_1^{(2)} \cdot x_j^{(1)} \left(1 - x_j^{(1)}\right), \quad j = 1, 2,$$

$$\delta_1^{(1)} = w_{11}^{(2)} \delta_1^{(2)} \cdot x_1^{(1)} \left(1 - x_1^{(1)}\right) = 2 \times 0.1439 \times 0.5 \times 0.5 \approx 0.0720$$

$$\delta_2^{(1)} = w_{21}^{(2)} \delta_1^{(2)} \cdot x_2^{(1)} \left(1 - x_2^{(1)}\right) = (-1) \times 0.1439 \times 0.88 \times 0.12 \approx -0.0152$$



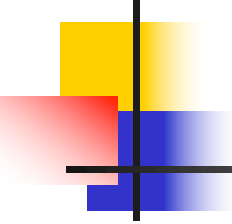
6.4.3 反向传播算法

■ 例题

- **题 7**: 学习率 $\rho = 0.5$ 。已知某参数当前值及其梯度如下:

参数	当前值	梯度
$w_{11}^{(2)}$	2	0.1
$w_{21}^{(2)}$	-1	-0.2
$b^{(2)}$	0.5	0.4

- 求更新后的参数值: $w_{11}^{(2)} = \underline{\hspace{1cm}}$, $w_{21}^{(2)} = \underline{\hspace{1cm}}$, $b^{(2)} = \underline{\hspace{1cm}}$ 。



6.4.3 反向传播算法

■ 例题解答

■ 题 7:

$$w_{11}^{(2)} \leftarrow w_{11}^{(2)} - \rho \cdot \frac{\partial E}{\partial w_{11}^{(2)}} = 2 - 0.5 \times 0.1 = 2 - 0.05 = 1.95$$

$$w_{21}^{(2)} \leftarrow w_{21}^{(2)} - \rho \cdot \frac{\partial E}{\partial w_{21}^{(2)}} = -1 - 0.5 \times (-0.2) = -1 + 0.1 = -0.9$$

$$b^{(2)} \leftarrow b^{(2)} - \rho \cdot \frac{\partial E}{\partial b^{(2)}} = 0.5 - 0.5 \times 0.4 = 0.5 - 0.2 = 0.3$$